HOUSEHOLD ELECTRICITY CONSUMPTION PREDICTION SYSTEM

A PROJECT REPORT

Submitted by

Kunal Mittal (24BCS12354)
Ansh Kumar (24BCS12097)
Vishal Kumar Shrivastava (24BCS12629)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE & ENGINEERING

Chandigarh University

November 2026

BONAFIDE CERTIFICATE

Certified that this project report "Household Electricity Consumption Prediction System" is the bonafide work of "Kunal Mittal, Ansh Kumar and Vishal Kumar Shrivastava" who carried out the project work under my/our supervision.

SIGNATURE **SIGNATURE**

HEAD OF THE DEPARTMENT **SUPERVISOR**

Submitted for the project viva-voce examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

TABLE OF CONTENTS

ABSTRACT 8
GRAPHICAL ABSTRACT 9
ABBREVIATIONS 10
SYMBOLS 11
CHAPTER 1. INTRODUCTION 12
1.1. Identification of Client / Need / Relevant Contemporary issue 12
1.2. Identification of Problem 12
1.3. Identification of Tasks 13
1.4. Timeline 13
1.5. Organization of the Report 13
CHAPTER 2. LITERATURE REVIEW/BACKGROUND STUDY 15
2.1. Timeline of the reported problem 15
2.2. Existing solutions 15
2.3. Bibliometric analysis 16
2.4. Review Summary 16
2.5. Problem Definition 16
2.6. Goals/Objectives 17
CHAPTER 3. DESIGN FLOW/PROCESS 18
3.1. Evaluation & Selection of Specifications/Features 18
3.2. Design Constraints 19
3.3. Analysis of Features and finalization subject to constraints 20
3.4. Design Flow 20
3.5. Design selection 21
3.6. Implementation plan/methodology 21
CHAPTER 4. RESULTS ANALYSIS AND VALIDATION 24
4.1. Implementation of solution 24
CHAPTER 5. CONCLUSION AND FUTURE WORK 29
5.1. Conclusion 29
5.2. Future work 29
REFERENCES 31
APPENDIX 33

1. Plagiarism Report 33
2. Design Checklist 33
USER MANUAL 34

List of Figures

Figure 0.1*: Graphical abstract: end-to-end MLOps architecture of the system* 9
Figure 1.1*: Gantt chart of the project timeline* 13
Figure 3.1*: Daily mean active power with 30-day rolling mean (strong yearly seasonality)* 18
Figure 3.2*: Average consumption by weekday and hour (morning and evening peaks)* 18
Figure 3.3*: Monthly pattern and share of energy per sub-meter* 19
Figure 3.4*: Alternative designs considered* 20
Figure 3.5*: Flowchart of the training and deployment algorithm* 22
Figure 3.6*: Detailed block diagram of the MLOps architecture* 22
Figure 4.1*: Correlation matrix of the measured variables* 24
Figure 4.2*: Validation MAE and R² of candidate models (best in orange)* 25
Figure 4.3*: Next-hour forecast vs actual consumption, last 14 days of the test set* 26
Figure 4.4*: Predicted vs actual scatter and residual distribution* 26
Figure 4.5*: Mean absolute error by hour of day on the test set* 26
Figure 4.6*: Top 15 features by permutation importance* 27
Figure 4.7*: Population Stability Index of monitored features (train vs test)* 28
Figure M.1*: Interactive API documentation (Swagger UI) at /api/docs* 35
Figure M.2*: Streamlit dashboard – Forecast tab* 36
Figure M.3*: Streamlit dashboard – Data explorer tab* 36
Figure M.4*: Streamlit dashboard – Model performance tab* 37
Figure M.5*: Streamlit dashboard – Monitoring tab* 37
Figure M.6*: Streamlit dashboard – MLOps pipeline tab* 38

List of Tables

Table 0.1*: Standards referred to in the project* 7
Table 0.2*: List of abbreviations* 10
Table 0.3*: List of symbols* 11
Table 1.1*: Identified tasks* 13
Table 2.1*: Timeline of the reported problem* 15
Table 2.2*: Bibliometric comparison of existing solutions* 16
Table 3.1*: Selected feature specification (31 features)* 19
Table 3.2*: Design constraints* 19
Table 3.3*: Comparison of alternative designs* 21
Table 3.4*: Software modules of the implementation* 23
Table 4.1*: Validation performance of candidate models (Jan–May 2010)* 24
Table 4.2*: Held-out test performance (Jun–Nov 2010)* 25
Table 4.3*: Automated test suite results* 27
Table 4.4*: Achievement of objectives* 28
Table A.1*: Design checklist* 33

List of Standards

Table 0.1*: Standards referred to in the project*

Standard	Publishing Agency	About the standard	Where used
ISO/IEC 23053:2022	ISO / IEC	Framework for Artificial Intelligence systems using Machine Learning; defines the ML pipeline components (data acquisition, pre-processing, training, evaluation, deployment).	Ch. 3.2, 3.6
ISO/IEC 25010:2023	ISO / IEC	Systems and software quality model (functional suitability, reliability, maintainability, portability).	Ch. 3.2, 4.1
IEC 62056 (DLMS/COSEM)	IEC	Electricity metering data exchange; defines how smart meters expose load-profile readings such as those used as model inputs.	Ch. 3.1
IS 16444	Bureau of Indian Standards	A.C. static direct-connected smart energy meters; relevant for deploying the system on Indian household smart meters.	Ch. 3.2
ISO 50001:2018	ISO	Energy management systems: requirements for measuring, analysing and improving energy performance.	Ch. 1.1, 3.2
OpenAPI Specification 3.1	OpenAPI Initiative	Language-agnostic description of REST APIs; the prediction service publishes an OpenAPI 3.1 schema.	Ch. 4.1

ABSTRACT

Residential buildings account for a large share of national electricity demand, yet most households receive only a monthly bill and have no forward-looking view of their consumption. Utilities, in turn, need short-term load forecasts at the household level to plan demand-response programmes as smart meters are rolled out. This project develops **VoltCast**, a Household Electricity Consumption Prediction System that forecasts a household's hourly active power for the next 1–48 hours and is engineered end to end following Machine Learning Operations (MLOps) practice.

The system uses the public UCI "Individual Household Electric Power Consumption" dataset: 2,075,259 one-minute readings recorded over almost four years in a house near Paris. A modular, configuration-driven pipeline ingests and validates the data, cleans 1.252% missing values, resamples to 34,589 hourly records and engineers 31 leakage-free lag, rolling-window, calendar and sub-meter features. Five models (persistence and seasonal-naive baselines, Ridge regression, Random Forest and Histogram Gradient Boosting) are trained on a strict chronological split and tracked in MLflow. The best model, Histogram Gradient Boosting, reaches a test MAE of 0.293 kW, RMSE of 0.432 kW and R^2 of 0.623, improving MAE by 22.2% and RMSE by 24.7% over the persistence baseline.

The model is registered in the MLflow model registry, served through a containerised FastAPI REST service with input validation and prediction intervals, and presented in an interactive Streamlit dashboard. A GitHub Actions workflow re-runs the pipeline, enforces a quality gate, executes the automated test suite and builds the Docker image on every change and on a weekly schedule. A monitoring stage computes the Population Stability Index and rolling error; it correctly detected seasonal drift between the winter-heavy training data and the summer test period and raised a retraining recommendation, demonstrating the closed MLOps loop.

Keywords: load forecasting, smart meter, gradient boosting, MLOps, MLflow, FastAPI, data drift.

GRAPHICAL ABSTRACT

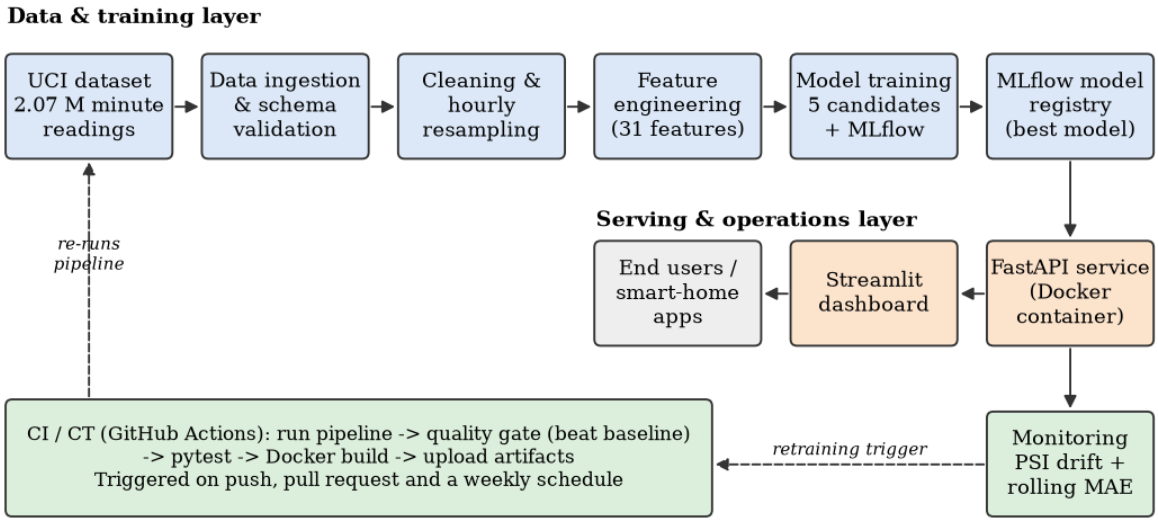


Figure 0.1*: Graphical abstract: end-to-end MLOps architecture of the system*

ABBREVIATIONS

Table 0.2*: List of abbreviations*

Abbreviation	Full form
AI	Artificial Intelligence
API	Application Programming Interface
ARIMA	Auto-Regressive Integrated Moving Average
CI/CD	Continuous Integration / Continuous Delivery
CT	Continuous Training
CNN	Convolutional Neural Network
DVC	Data Version Control
EDA	Exploratory Data Analysis
HGB	Histogram-based Gradient Boosting
IEA	International Energy Agency
kW / kWh	Kilowatt / Kilowatt-hour
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
ML	Machine Learning
MLOps	Machine Learning Operations
NILM	Non-Intrusive Load Monitoring
PSI	Population Stability Index
REST	Representational State Transfer
RF	Random Forest
RMSE	Root Mean Squared Error
R^2	Coefficient of determination
SVR	Support Vector Regression
UCI	University of California, Irvine (ML Repository)

SYMBOLS

Table 0.3*: List of symbols*

Symbol	Meaning
y_t	Hourly mean global active power at hour t (kW)
$\hat{y}_t$	Predicted active power at hour t (kW)
lag_k	Value of the target k hours earlier, y_{t-k}
μ_w, σ_w	Rolling mean and standard deviation over the previous w hours
n	Number of evaluation samples
α	Ridge regularisation strength
η	Learning rate of gradient boosting
σ_r	Standard deviation of test residuals (used for prediction intervals)
h	Forecast horizon in hours
p_i, q_i	Share of reference and current data in bin i (PSI)

CHAPTER 1.
INTRODUCTION

1.1. Identification of Client / Need / Relevant Contemporary issue

Electricity use in buildings is one of the largest and fastest-growing components of energy demand. The International Energy Agency reports that buildings account for roughly 30% of global final energy consumption, and space cooling, water heating and appliances in homes are major drivers of evening peak demand [1]. In India, the domestic (household) sector consumes about one quarter of all electricity sold, and its share keeps rising with appliance ownership and air-conditioner adoption [2].

Three groups of stakeholders need short-term household consumption forecasts:

- **Households** want to know how much energy and money they will use tomorrow, so they can shift flexible loads (washing machines, water heaters) and avoid bill shock; time-of-day tariffs make this more valuable.
- **Distribution utilities** run demand-response and peak-shaving programmes and must forecast load at fine granularity. India's Revamped Distribution Sector Scheme targets the installation of about 250 million prepaid smart meters, which will generate exactly the kind of high-frequency data this project uses [3].
- **Smart-home and energy-management vendors** need a deployable, monitored prediction service rather than a one-off notebook model, which is the gap that MLOps practice addresses [4], [5].

Studies of production ML systems show that the model code is a small part of a real ML system and that most failures come from data dependencies, configuration and lack of monitoring [4]. A consumption predictor that silently degrades when seasons or occupants change is a contemporary, documented issue.

1.2. Identification of Problem

Households and utilities lack a reliable, continuously maintained way to anticipate a single home's electricity demand over the next hours. Household load is highly volatile, depends on human routines, weekday, season and appliance usage, and drifts over time. Meter readings contain gaps and noise. Models built once and left unattended become stale, and there is no feedback that tells the operator when this happens.

1.3. Identification of Tasks

The work was divided into the following tasks, which also define the structure of this report:

Table 1.1*: Identified tasks*

#	Task	Output	Chapter
T1	Survey of the problem, existing forecasting methods and MLOps literature	Problem definition, objectives	1, 2
T2	Selection of a public, realistic smart-meter dataset and EDA	Data understanding, figures	3.1, 4.1
T3	Design of features, constraints and alternative system designs	Selected design	3
T4	Data pipeline: ingestion, validation, cleaning, resampling, feature engineering	Versioned datasets	3.6
T5	Model training, experiment tracking, selection and registry	Registered model	4.1
T6	Deployment: REST API, Docker, dashboard	Running services	4.1
T7	Automation and quality: CI/CT workflow, automated tests, monitoring	CI pipeline, drift report	4.1
T8	Validation, documentation, report and user manual	This report	4, 5, Manual

1.4. Timeline

The project was executed over twelve weeks. Figure 1.1 shows the Gantt chart; tasks overlap where the pipeline allowed parallel work (for example, the dashboard was built while CI was configured).

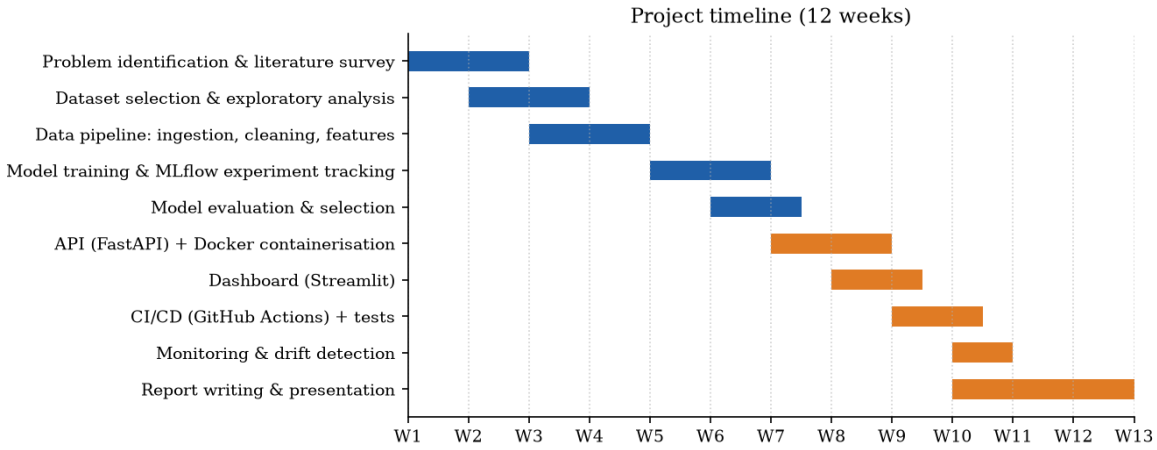


Figure 1.1*: Gantt chart of the project timeline*

1.5. Organization of the Report

- **Chapter 1** introduces the need, the problem, the tasks and the timeline.

- **Chapter 2** reviews the history of load forecasting, existing statistical, machine-learning and deep-learning solutions, compares them, and defines the problem and objectives.
- **Chapter 3** evaluates features and constraints, compares two alternative designs, selects one and presents the implementation methodology.
- **Chapter 4** presents the implementation and validates the results with metrics, plots, tests and monitoring outputs.
- **Chapter 5** concludes, discusses deviations from expectations and lists future work.
- References, Appendix and a step-by-step User Manual follow.

CHAPTER 2.
LITERATURE REVIEW/BACKGROUND STUDY

2.1. Timeline of the reported problem

Table 2.1*: Timeline of the reported problem*

Period	Milestone
1970s	Box–Jenkins ARIMA methodology becomes the standard for utility-level load forecasting [6].
1992	Hart introduces Non-Intrusive Load Monitoring, showing household appliance loads can be inferred from aggregate meter data [7].
2000s	Machine-learning regressors (SVR, Random Forest [8], Gradient Boosting [9]) applied to building energy prediction.
2012	UCI publishes the Individual Household Electric Power Consumption dataset, which becomes a benchmark for household-level forecasting [10].
2015	Sculley et al. document "hidden technical debt" in production ML systems [4].
2016–2019	Deep learning (LSTM, CNN-LSTM) applied to household load forecasting [11], [12]; reviews of data-driven building energy prediction published [13].
2018	MLflow released to manage the ML lifecycle (tracking, projects, registry) [14].
2016–present	Probabilistic forecasting becomes the norm for load forecasting [15]; national smart-meter roll-outs (e.g. India's RDSS) make household data abundant [3].
2023	MLOps formally defined and its architecture surveyed in IEEE Access [5].

2.2. Existing solutions

Statistical time-series models. ARIMA and exponential smoothing [6], [16] model linear autocorrelation and seasonality. They are interpretable and cheap but assume stationarity and struggle with the abrupt, appliance-driven spikes of a single household.

Classical machine learning. Fan et al. combined data-mining techniques to predict next-day building consumption and peak demand [17]. Tree ensembles such as Random Forest [8] and gradient boosting [9], [18] learn non-linear interactions from engineered lag and calendar features and are robust to scale and outliers.

Deep learning. Marino et al. applied sequence-to-sequence LSTMs to the same UCI dataset [11]; Kim and Cho combined CNN and LSTM layers and reported lower error than linear and LSTM baselines [12]. These models capture long dependencies but require more data, GPU training and careful tuning.

MLOps platforms. Kreuzberger et al. describe the components of a production ML system: versioned data and code, experiment tracking, model registry, automated pipelines,

serving and monitoring [5]. MLflow [14] and drift-detection methods [19] are widely used building blocks.

2.3. Bibliometric analysis

Table 2.2*: Bibliometric comparison of existing solutions*

Work	Approach	Key features	Effectiveness	Drawbacks
Box & Jenkins [6]	ARIMA	Linear autoregression, differencing	Good for aggregate, smooth loads	Poor on volatile single-house data; manual order selection
Fan et al. [17]	Ensemble data mining	Feature selection + multiple regressors	Accurate next-day building forecasts	Commercial buildings; no deployment
Marino et al. [11]	LSTM seq2seq	Learns temporal patterns directly	Competitive on UCI data at hourly scale	Overfits at minute scale; heavy training
Kim & Cho [12]	CNN-LSTM	Convolutional feature extraction + LSTM	Low error on UCI dataset	Black box; GPU cost; no monitoring
Amasyali & El-Gohary [13]	Review	Survey of 60+ studies	Shows ML methods outperform statistical ones in most cases	Notes lack of deployed, maintained systems
Kreuzberger et al. [5]	MLOps architecture	CI/CD/CT, registry, monitoring	Reference architecture	Not specific to energy forecasting

2.4. Review Summary

The literature shows that (i) machine-learning models with good lag and calendar features consistently beat purely statistical models on household data; (ii) deep networks can be marginally more accurate but at much higher training and operational cost; and (iii) almost all published household forecasting work stops at an offline accuracy number, with no serving, automation or monitoring. This project therefore combines a strong, efficient tree-based model with a complete MLOps pipeline, and evaluates it honestly against naive baselines on a chronological hold-out.

2.5. Problem Definition

What is to be done: build a system that, given the most recent week of hourly smart-meter readings for a household, forecasts the household's mean active power (kW) for each of the

next 1–48 hours with a prediction interval, and that can be retrained, tested, deployed and monitored automatically.

How it is to be done: using the UCI household dataset, a reproducible configuration-driven pipeline, supervised regression models compared under MLflow tracking, a FastAPI service packaged with Docker, a Streamlit dashboard, a GitHub Actions CI/CT workflow and PSI-based drift monitoring.

What is not to be done: appliance-level disaggregation (NILM), control of household devices, billing integration, or collection of new personal data. Forecasts are for a single household and are not intended for grid-level dispatch.

2.6. Goals/Objectives

1. Clean and resample the 2.07 M-row dataset into a gap-free hourly series with an auditable data report.
2. Engineer at least 25 features that use only past information (verified by an automated leakage test).
3. Train and compare at least four models under experiment tracking on a strict chronological split.
4. Beat the persistence baseline on the held-out test period by at least 15% in MAE.
5. Serve forecasts through a documented REST API with input validation and a response time under 1 s.
6. Automate pipeline, quality gate, tests and container build in a CI/CT workflow.
7. Detect data drift (PSI > 0.2) and performance degradation, and recommend retraining automatically.

CHAPTER 3.
DESIGN FLOW/PROCESS

3.1. Evaluation & Selection of Specifications/Features

Dataset. The UCI dataset [10] contains 2,075,259 one-minute measurements from 2006-12-16 to 2010-11-26 for one house in Sceaux, France: global active and reactive power, voltage, current intensity and three sub-meters (kitchen; laundry room; electric water heater and air conditioner). It was chosen because it is public, long enough to cover four winters, contains real gaps, and mirrors the load-profile channels exposed by IEC 62056 smart meters. Figures 3.1–3.3 summarise the exploratory analysis.

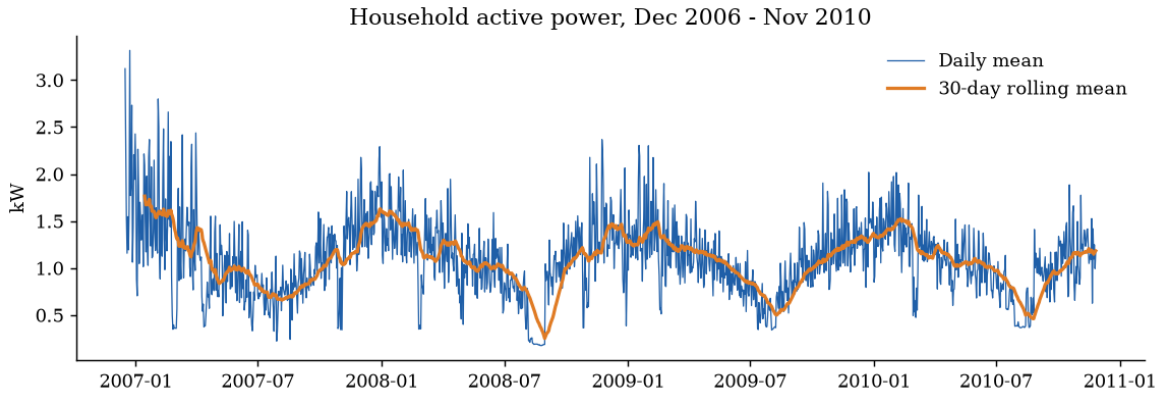


Figure 3.1*: Daily mean active power with 30-day rolling mean (strong yearly seasonality)*

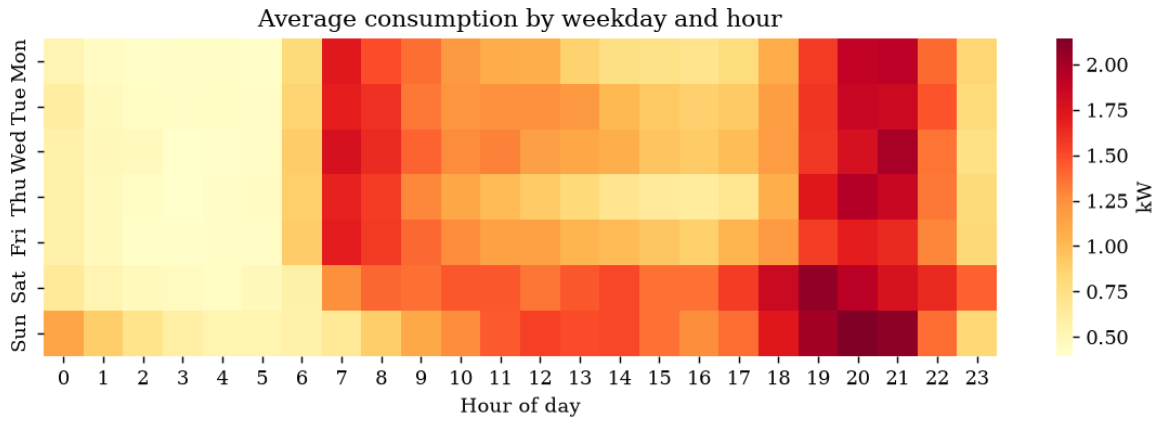


Figure 3.2*: Average consumption by weekday and hour (morning and evening peaks)*

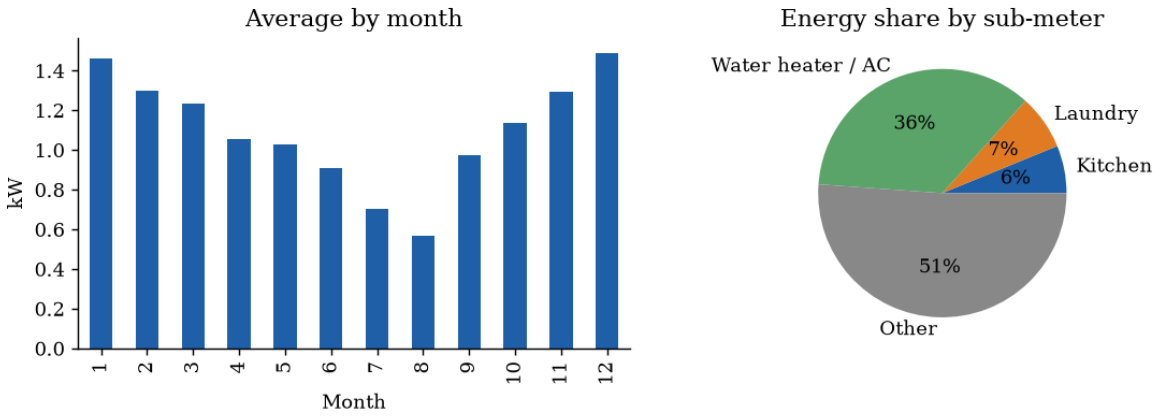


Figure 3.3*: Monthly pattern and share of energy per sub-meter*

The EDA shows a strong winter peak (consumption roughly doubles from August to December), two daily peaks around 07:00 and 20:00–21:00, higher daytime use at weekends, and an August holiday dip. These findings drove the feature specification in Table 3.1.

Table 3.1*: Selected feature specification (31 features)*

Feature group	Features	Rationale
Autoregressive lags	lag 1, 2, 3, 6, 12, 24, 48, 168 h	Short-term persistence, same hour yesterday and last week
Rolling statistics	mean and std over 3, 6, 24, 168 h	Recent level and volatility
Calendar	hour, weekday, month, weekend, French public holiday	Routines and seasonality
Cyclical encodings	sin/cos of hour and month	23:00 and 00:00 are neighbours
Electrical context (t−1)	reactive power, voltage, intensity, 3 sub-meters	Which appliances were active

3.2. Design Constraints

Table 3.2*: Design constraints*

Constraint	Consideration in the design
Standards	ML life-cycle stages follow ISO/IEC 23053; software quality attributes of ISO/IEC 25010 (reliability, maintainability) drive testing and modularity; API documented to OpenAPI 3.1.
Economic / Cost	Must run on a student laptop and free CI runners: no GPU, training under one minute, only open-source tools (scikit-learn, MLflow, FastAPI, Streamlit, Docker).
Environmental	Low-compute models reduce the energy footprint of retraining; the purpose of the system is itself to support energy saving (ISO 50001 spirit).
Health & Safety	Advisory system only: it never switches appliances, so incorrect forecasts cannot cause physical harm.
Professional / Ethical	Honest evaluation against naive baselines on unseen future data; no data leakage; limitations are disclosed.
Social & Privacy	Fine-grained meter data can reveal occupancy and routines. The system

	uses a public anonymised dataset, stores no personal identifiers and processes data on-premise.
Manufacturability / Deployability	Must be reproducible on any machine: pinned configuration, Docker image, single-command pipeline.
Regulatory	Indian smart meters follow IS 16444; the API input schema maps to the hourly load profile such meters provide.

3.3. Analysis of Features and finalization subject to constraints

- **Removed:** minute-level forecasting. At one-minute resolution the series is dominated by appliance switching noise; hourly resolution matches billing and demand-response use and reduces data 60×.
- **Removed:** weather features. They would improve accuracy but no free, historical, hourly weather feed for Sceaux 2006–2010 could be integrated within the cost constraint.
- **Removed:** deep LSTM model as the production candidate, due to the no-GPU and explainability constraints (kept as future work).
- **Modified:** missing data handling. Gaps up to 6 h are time-interpolated; longer outages (e.g. multi-day gaps in 2010) are filled with the same hour of the previous week to preserve the daily profile.
- **Added:** prediction intervals ($\pm 1.96\sigma_r\sqrt{h}$) so users can see forecast uncertainty.
- **Added:** automated leakage test and a baseline quality gate to satisfy the professional/ethical constraint.

3.4. Design Flow

Two alternative designs were developed to meet the specification (Figure 3.4).

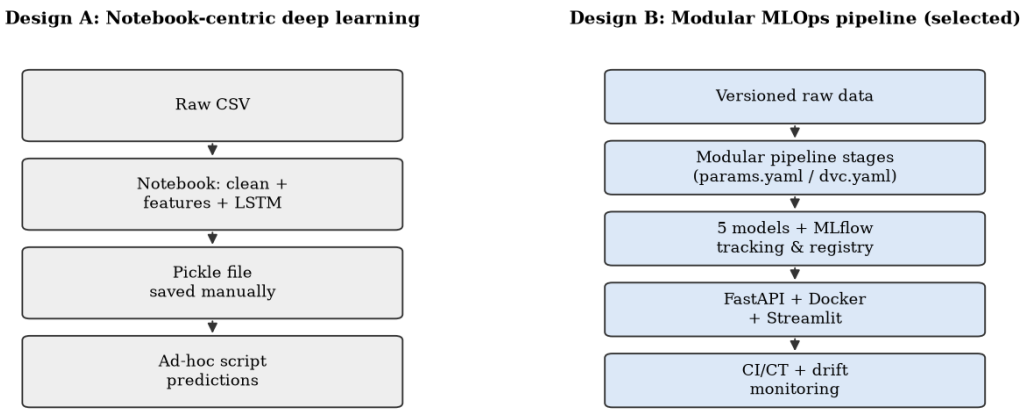


Figure 3.4*: Alternative designs considered*

Design A – Notebook-centric deep learning. All steps (cleaning, features, an LSTM model) live in a single Jupyter notebook. The trained network is pickled manually and predictions are produced with an ad-hoc script. This is the typical approach in the literature [11], [12].

Design B – Modular MLOps pipeline. Each stage is an independent, tested Python module driven by one configuration file (params.yaml) and declared as a DVC pipeline. Several models are compared under MLflow tracking; the best is registered and served by a Dockerised FastAPI service and a Streamlit dashboard; GitHub Actions automates the pipeline and a monitoring stage closes the loop.

3.5. Design selection

Table 3.3*: Comparison of alternative designs*

Criterion	Design A	Design B
Expected accuracy	High (with tuning)	High (gradient boosting)
Training cost / hardware	GPU preferred, minutes–hours	CPU, < 10 s per model
Reproducibility	Low: hidden notebook state	High: config + pipeline + Docker
Experiment tracking	Manual	Automatic (MLflow)
Deployment	Manual script	REST API + container + dashboard
Testing / CI	None	8 automated tests, CI quality gate
Monitoring & retraining	None	PSI drift + MAE checks, weekly CT
Explainability	Low	Permutation importance
Fit with constraints	Violates cost & reproducibility	Satisfies all

Design B is selected. It meets every constraint in Section 3.2, its accuracy is competitive, and it delivers the operational capabilities (automation, deployment, monitoring) that are the core learning outcomes of an MLOps project and that the literature identifies as missing.

3.6. Implementation plan/methodology

Figure 3.5 shows the algorithm executed by the pipeline and Figure 3.6 the detailed block diagram.

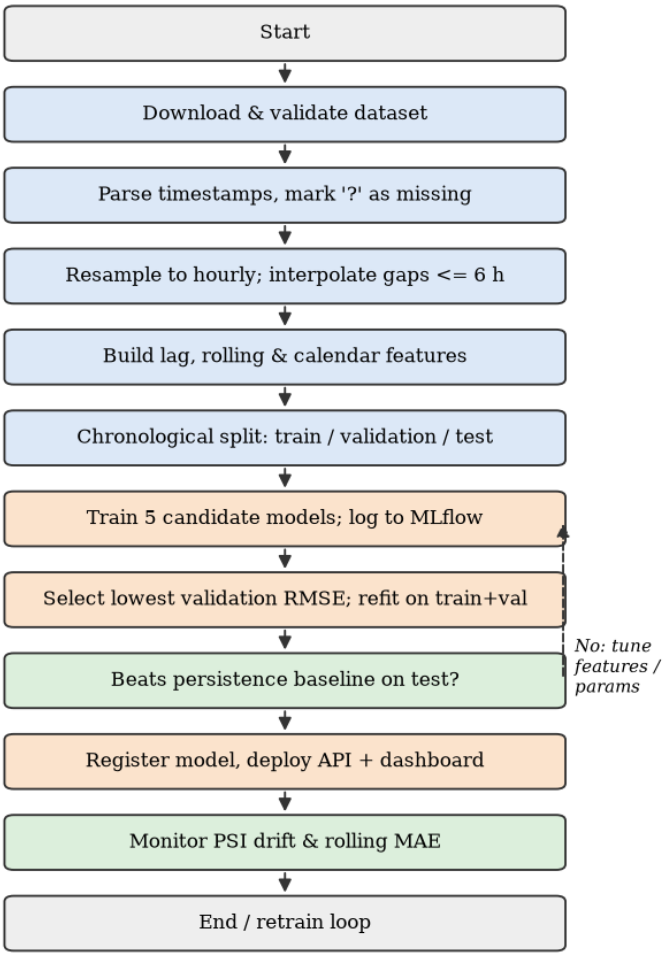


Figure 3.5*: Flowchart of the training and deployment algorithm*

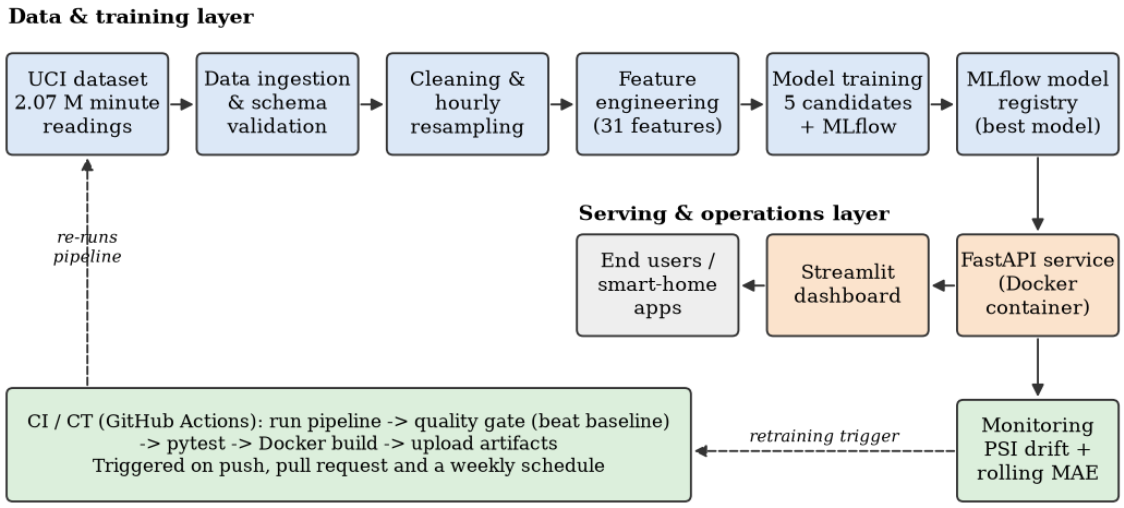


Figure 3.6*: Detailed block diagram of the MLOps architecture*

Data split. To respect time ordering, data is split chronologically: training up to 31 Dec 2009 (26,503 h), validation Jan–May 2010 (3,624 h) and test 2010-06-01 to 2010-11-26

(4,294 h). Models are selected on validation, refit on train+validation, and scored once on test.

Models. Persistence ($\hat{y}_t = y_{t-1}$) and seasonal-naive ($\hat{y}_t = y_{t-24}$) baselines; Ridge regression ($\alpha = 1$, standardised inputs); Random Forest (200 trees, depth 16) [8]; Histogram Gradient Boosting (500 iterations, $\eta = 0.05$, 31 leaves) [9], [18].

Metrics. MAE = $(1/n)\Sigma|y_t - \hat{y}_t|$, RMSE = $\sqrt{((1/n)\Sigma(y_t - \hat{y}_t)^2)}$, R^2 and MAPE. RMSE is used for model selection because peak errors matter most for demand response.

Monitoring. PSI = $\Sigma(q_i - p_i)\cdot\ln(q_i/p_i)$ over 10 quantile bins of the training distribution; PSI > 0.2 signals significant drift. Retraining is recommended when any key feature drifts or recent 30-day MAE degrades more than 20%.

Table 3.4*: Software modules of the implementation*

Module	Responsibility
src/data_ingestion.py	Download dataset, verify schema
src/preprocess.py	Parse, clean, derive unmetered energy, resample hourly, fill gaps
src/features.py	Build 31 leakage-free features
src/train.py	Train/compare 5 models, MLflow logging, model registry, metadata
src/monitor.py	PSI drift and performance degradation, retraining decision
src/evaluate.py	EDA and diagnostic figures, permutation importance
src/predict.py	Recursive multi-step forecasting with intervals
serving/main.py, app.py	FastAPI service: /predict, /forecast, /health, /model-info, /metrics
web/ (React + Vite), src/export_web.py	React web dashboard hosted on Vercel and its data export
dashboard/app.py	Streamlit dashboard (5 tabs)
tests/test_pipeline.py	Unit, leakage, API and quality-gate tests
.github/workflows/ci.yml, Dockerfile, dvc.yaml	Automation, packaging, reproducibility

CHAPTER 4.
RESULTS ANALYSIS AND VALIDATION

4.1. Implementation of solution

Modern tools used. Analysis: pandas, NumPy, Matplotlib, Seaborn, Plotly. Design drawings: Matplotlib-generated architecture, flow and Gantt diagrams. Modelling: scikit-learn. Experiment management: MLflow (SQLite backend and model registry). Project management and communication: Git, GitHub Actions, DVC pipeline definition. Testing and validation: pytest, FastAPI TestClient, PSI drift analysis. Report preparation: python-docx generated directly from pipeline outputs, so every number in this chapter is reproducible.

Data preparation results. 1.252% of minute readings were missing ('?'). After cleaning and hourly resampling the series has 34,589 gap-free records with mean 1.09 kW (median 0.80 kW, max 6.56 kW). Feature building drops the first 168 h (needed for the weekly lag), leaving 34,421 modelling rows. Figure 4.1 shows that current intensity is almost perfectly correlated with active power, while voltage is weakly negatively correlated.

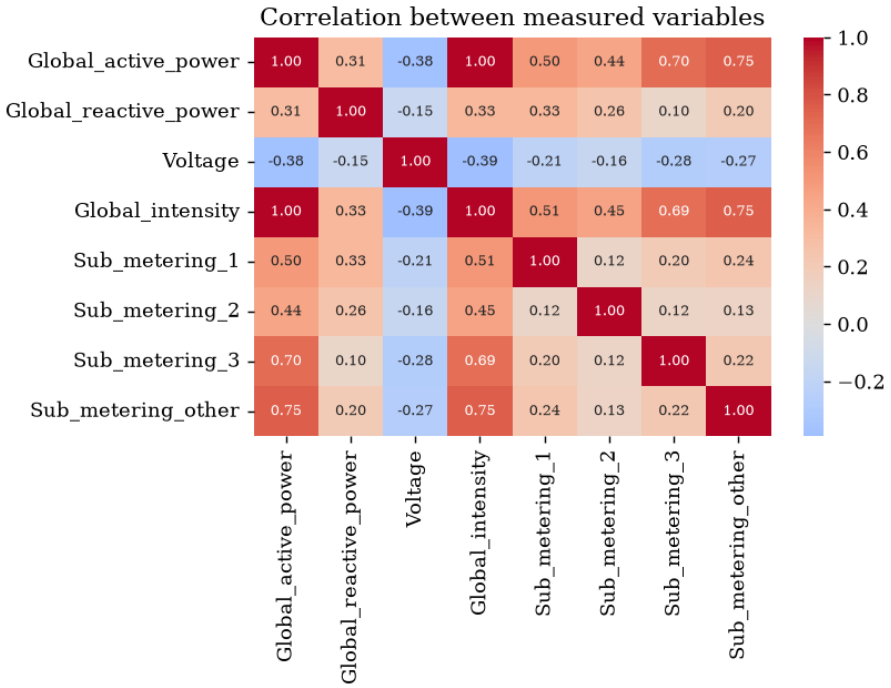


Figure 4.1*: Correlation matrix of the measured variables*

Model comparison. Table 4.1 and Figure 4.2 show validation results tracked in MLflow.

Table 4.1*: Validation performance of candidate models (Jan–May 2010)*

Model	MAE (kW)	RMSE (kW)	R^2	MAPE (%)	Fit time (s)
Persistence Baseline	0.4414	0.6686	0.3635	43.3	0.00
Seasonal Naive 24H	0.6359	0.9121	-0.1845	72.6	0.00
Ridge Regression	0.3980	0.5568	0.5586	44.4	0.02
Random Forest	0.3451	0.5087	0.6316	37.2	3.86
Hist Gradient Boosting	**0.3415**	**0.4982**	**0.6466**	**36.7**	**0.63**

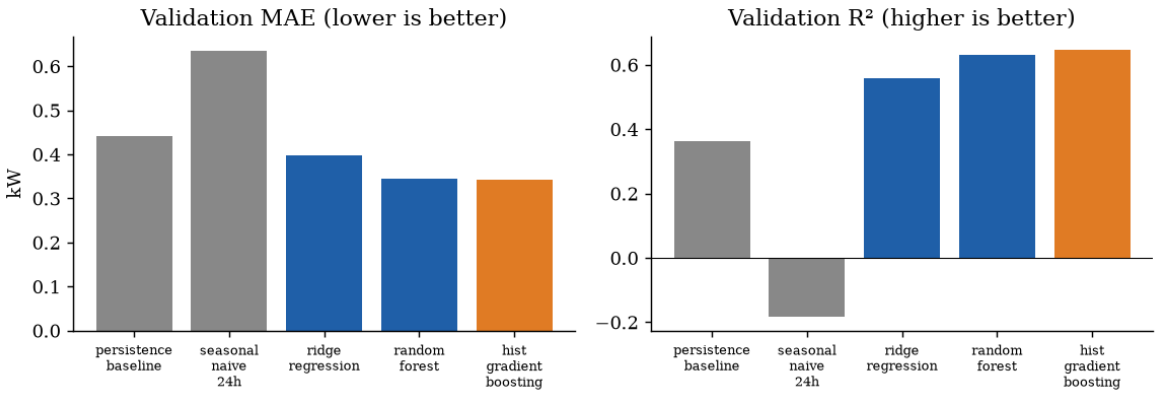


Figure 4.2*: Validation MAE and R^2 of candidate models (best in orange)*

Both ML ensembles clearly beat the baselines and the linear model. Histogram Gradient Boosting has the lowest validation RMSE (0.4982 kW) and trains six times faster than Random Forest, so it was selected. The seasonal-naive model has negative R^2: yesterday's same hour is a worse predictor than the mean, confirming that household load is driven by irregular routines.

Test-set results. The selected model was refit on train+validation and evaluated once on the untouched test period (Table 4.2).

Table 4.2*: Held-out test performance (Jun–Nov 2010)*

Model	MAE (kW)	RMSE (kW)	R^2	MAPE (%)
Persistence baseline	0.3773	0.5735	0.3349	46.9
Histogram Gradient Boosting	**0.2934**	**0.4320**	**0.6227**	**39.5**
Improvement	22.2%	24.7%	+0.288	–

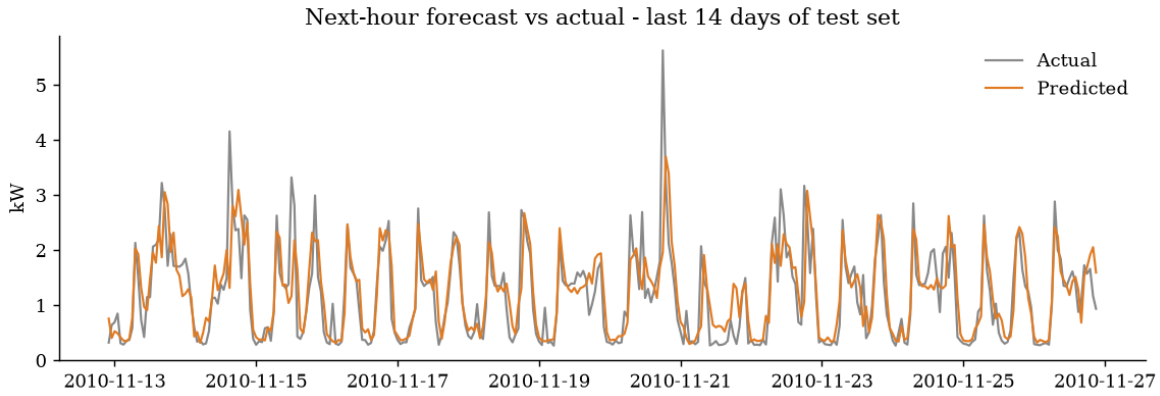


Figure 4.3: *Next-hour forecast vs actual consumption, last 14 days of the test set*

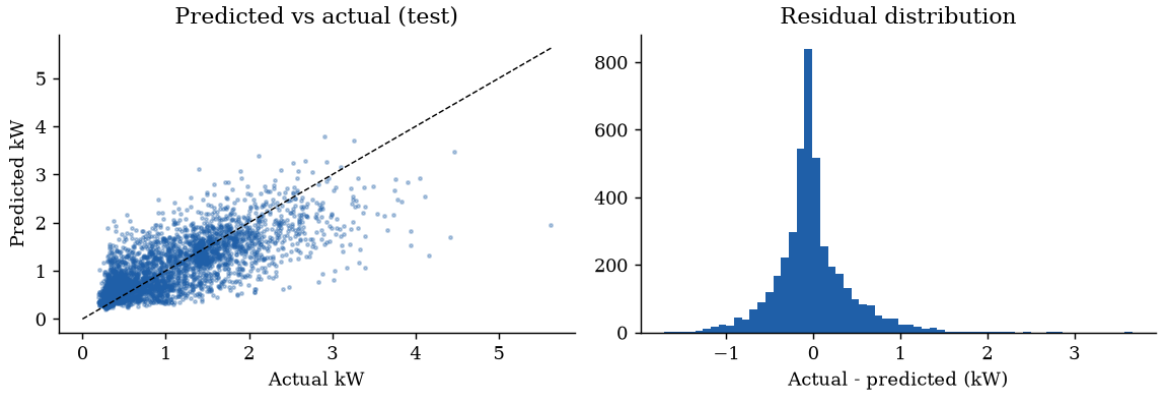


Figure 4.4: *Predicted vs actual scatter and residual distribution*

The forecasts follow the daily cycle and the timing of evening peaks closely (Figure 4.3). Residuals are centred near zero but right-skewed (Figure 4.4): the largest errors are sudden short spikes, such as an oven or water heater switching on, which are inherently unpredictable from past meter data alone.

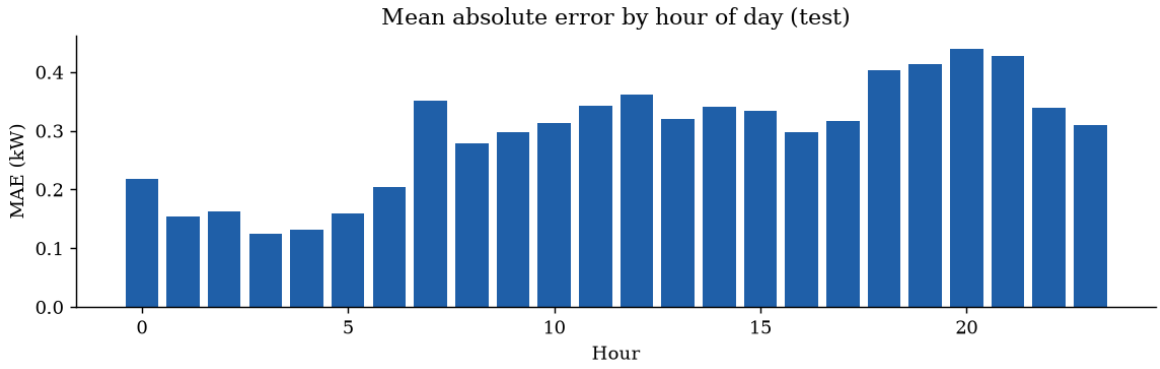


Figure 4.5: *Mean absolute error by hour of day on the test set*

Error is lowest at night (stable base load) and highest during the 07:00 and 19:00–22:00 activity peaks (Figure 4.5), where occupant behaviour dominates.

Explainability. Permutation importance (Figure 4.6) shows which inputs the model relies on.

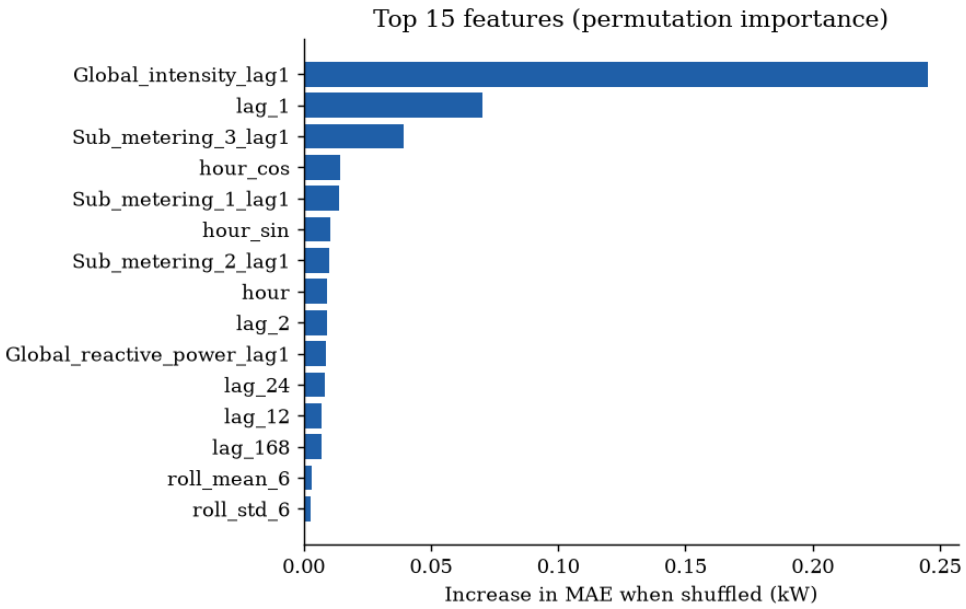


Figure 4.6*: Top 15 features by permutation importance*

The most important inputs are Global_intensity_lag1 (0.245), lag_1 (0.070), Sub_metering_3_lag1 (0.039), hour_cos (0.014). The previous hour's current intensity and power carry the household's current state, the water-heater/AC sub-meter signals heavy thermal loads, and the hour encodings capture daily routines, which is physically consistent.

Deployment validation. The FastAPI service and web dashboard are deployed on Vercel at https://voltcast-forecast.vercel.app, as a React single-page app backed by the FastAPI function, auto-deployed from the GitHub repository. The service was also run locally. A forecast request with one week of history returned a 24-hour forecast with intervals in well under one second. Interactive OpenAPI documentation is served at /docs (see User Manual).

Automated testing. The pytest suite (Table 4.3) passes in about 3 seconds and runs in CI on every push.

Table 4.3*: Automated test suite results*

Test	What it validates	Result
test_resample_fills_gaps	Missing hours are filled; output is gap-free	Pass
test_features_have_no_leakage	lag_1 and rolling means use only past values	Pass
test_metrics_perfect_prediction	Metric implementation correctness	Pass
test_psi_detects_shift	PSI < 0.1 for same distribution, > 0.2 for	Pass

	shifted	
test_health_endpoint	Service liveness and model loading	Pass
test_predict_endpoint_returns_horizon	Horizon length, interval ordering	Pass
test_predict_rejects_short_history	Input validation returns HTTP 422	Pass
test_model_beats_baseline	Quality gate: model MAE < baseline MAE	Pass

Monitoring validation. Figure 4.7 shows the PSI between the training and test distributions.

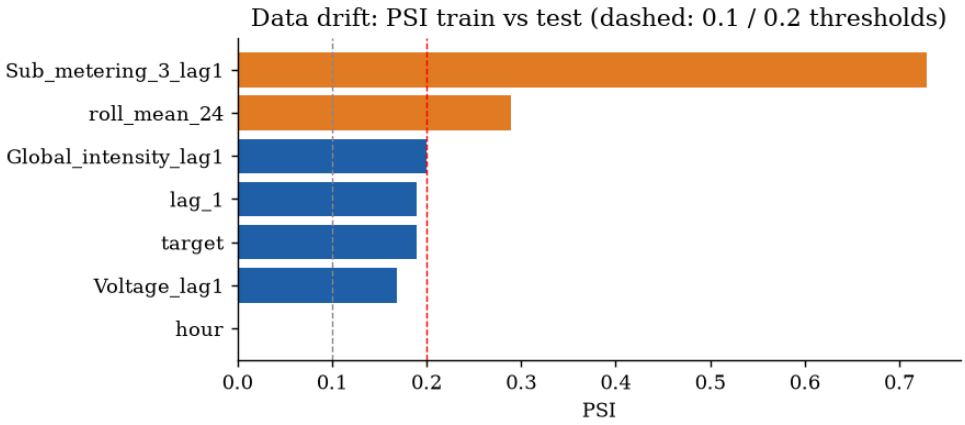


Figure 4.7*: Population Stability Index of monitored features (train vs test)*

Significant drift was detected in roll_mean_24, Sub_metering_3_lag1 (PSI 0.73 for the water-heater/AC sub-meter). This is genuine seasonal drift: the training data covers three winters, while the test period is June–November, when heating load is low. The 30-day rolling MAE (0.350 kW) was 19.2% above the reference test MAE as consumption climbed again in November. The monitor therefore set retraining_recommended = true, which is exactly the behaviour a production MLOps loop requires.

Table 4.4*: Achievement of objectives*

Objective (Section 2.6)	**Target**	**Achieved**
Clean hourly series	Gap-free	34,589 hourly rows, 0 gaps
Feature engineering	≥ 25 leakage-free	31 features, leakage test passes
Models compared	≥ 4	5 models tracked in MLflow
Improvement over baseline	≥ 15% MAE	22.2% MAE, 24.7% RMSE
REST API latency	< 1 s	Achieved (24-h forecast)
CI/CT automation	Pipeline + tests + build	GitHub Actions workflow
Drift detection	PSI > 0.2 flagged	2 features flagged, retrain advised

CHAPTER 5.
CONCLUSION AND FUTURE WORK

5.1. Conclusion

This project delivered VoltCast, a complete, working Household Electricity Consumption Prediction System built on MLOps principles. Starting from 2.07 million raw smart-meter readings, the automated pipeline produces a clean hourly series, 31 leakage-free features and five tracked models. The selected Histogram Gradient Boosting model forecasts next-hour consumption with a test MAE of 0.293 kW and R^2 of 0.623, a 22.2% MAE reduction over the persistence baseline. The model is registered in MLflow, served through a Dockerised FastAPI service, visualised in a Streamlit dashboard, protected by automated tests and a CI quality gate, and watched by a drift monitor.

Expected vs obtained results. The objective of a ≥ 15% improvement over the baseline was met. However, the absolute R^2 (~0.62) is lower than the >0.9 values sometimes reported in the literature. The reasons are: (i) we forecast a single household at hourly resolution, where behaviour-driven spikes are inherently unpredictable; (ii) we use a strict chronological split and refuse features computed from the hour being predicted, whereas some studies shuffle data or use same-hour intensity, which inflates scores; and (iii) the test period has a different seasonal distribution from training, which the monitor correctly flagged. MAPE (~39%) is inflated by very low night-time loads, where small absolute errors become large percentages; MAE and RMSE are the more meaningful measures here.

5.2. Future work

- **Weather and occupancy inputs:** add outdoor temperature (heating/cooling degree hours) and holiday calendars from an API; expected to reduce winter peak error substantially.
- **Deep and probabilistic models:** evaluate LSTM/Temporal Fusion Transformer and quantile gradient boosting for calibrated prediction intervals [12], [15].
- **Multi-household, Indian data:** train a global model across many homes using data from IS 16444 smart meters, with federated learning to protect privacy.
- **Automated retraining:** connect the monitor's retraining flag to a workflow trigger and promote a new model only if it beats the current one (champion/challenger) in the registry.

- **Cloud deployment:** deploy the container to a managed service with autoscaling, and add Prometheus/Grafana for service metrics.
- **Actionable recommendations:** combine forecasts with time-of-day tariffs to suggest the cheapest time to run flexible appliances.

REFERENCES

[1] International Energy Agency, “Buildings – Energy System,” IEA, Paris. [Online]. Available: https://www.iea.org/energy-system/buildings

[2] Central Electricity Authority, “General Review (All India Electricity Statistics),” Ministry of Power, Government of India, New Delhi.

[3] Ministry of Power, Government of India, “Revamped Distribution Sector Scheme (RDSS),” 2021. [Online]. Available: https://recindia.nic.in/revamped-distribution-sector-scheme

[4] D. Sculley et al., “Hidden technical debt in machine learning systems,” in Advances in Neural Information Processing Systems 28, 2015.

[5] D. Kreuzberger, N. Kühl and S. Hirschl, “Machine Learning Operations (MLOps): Overview, definition, and architecture,” IEEE Access, vol. 11, pp. 31866–31879, 2023.

[6] G. E. P. Box and G. M. Jenkins, Time Series Analysis: Forecasting and Control. San Francisco: Holden-Day, 1970.

[7] G. W. Hart, “Nonintrusive appliance load monitoring,” Proceedings of the IEEE, vol. 80, no. 12, pp. 1870–1891, 1992.

[8] L. Breiman, “Random forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[9] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001.

[10] G. Hebrail and A. Berard, “Individual Household Electric Power Consumption,” UCI Machine Learning Repository, 2012. doi: 10.24432/C58K54.

[11] D. L. Marino, K. Amarasinghe and M. Manic, “Building energy load forecasting using deep neural networks,” in Proc. IECON 2016 – 42nd Annual Conference of the IEEE Industrial Electronics Society, 2016, pp. 7046–7051.

[12] T.-Y. Kim and S.-B. Cho, “Predicting residential energy consumption using CNN-LSTM neural networks,” Energy, vol. 182, pp. 72–81, 2019.

[13] K. Amasyali and N. M. El-Gohary, “A review of data-driven building energy consumption prediction studies,” Renewable and Sustainable Energy Reviews, vol. 81, pp. 1192–1205, 2018.

[14] M. Zaharia et al., “Accelerating the machine learning lifecycle with MLflow,” IEEE Data Engineering Bulletin, vol. 41, no. 4, pp. 39–45, 2018.

[15] T. Hong and S. Fan, “Probabilistic electric load forecasting: A tutorial review,” International Journal of Forecasting, vol. 32, no. 3, pp. 914–938, 2016.

[16] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 3rd ed. Melbourne: OTexts, 2021.

[17] C. Fan, F. Xiao and S. Wang, “Development of prediction models for next-day building energy consumption and peak power demand using data mining techniques,” Applied Energy, vol. 127, pp. 1–10, 2014.

[18] G. Ke et al., “LightGBM: A highly efficient gradient boosting decision tree,” in Advances in Neural Information Processing Systems 30, 2017.

[19] S. Rabanser, S. Günnemann and Z. Lipton, “Failing loudly: An empirical study of methods for detecting dataset shift,” in Advances in Neural Information Processing Systems 32, 2019.

[20] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

APPENDIX

1. Plagiarism Report

2. Design Checklist

Table A.1*: Design checklist*

Item	Status
Problem identified and justified with documented sources	✓
Literature survey with comparison of existing solutions	✓
Measurable objectives defined	✓
Design constraints (standards, cost, ethics, privacy, safety) considered	✓
At least two alternative designs compared and one selected with reasons	✓
Flowchart / block diagram of implementation	✓
Public dataset, reproducible pipeline (params.yaml, dvc.yaml)	✓
Chronological train/validation/test split, baseline comparison	✓
Experiment tracking and model registry (MLflow)	✓
Model serving (FastAPI) and containerisation (Docker)	✓
Interactive dashboard (Streamlit)	✓
Automated tests and CI/CT workflow	✓
Data drift and performance monitoring	✓
User manual with step-by-step instructions	✓

USER MANUAL

Hosted version (no installation): open https://voltcast-forecast.vercel.app for the web dashboard and live forecasts, and https://voltcast-forecast.vercel.app/api/docs for the API. Source code: https://github.com/Kunal039/voltcast.

Prerequisites: Python 3.10 or newer, about 1 GB free disk space, internet access for the first run (dataset download, ~20 MB). Docker Desktop is optional.

Step 1: Get the project.

git clone https://github.com/Kunal039/voltcast.git then open a terminal inside the folder.

Step 2: Install dependencies.

pip install -r requirements-dev.txt

Step 3: Run the complete pipeline.

python run_pipeline.py (downloads data, preprocesses, builds features, trains and registers the model, runs monitoring and creates all figures; about 1 minute)

Step 4: Run the tests.

pytest -q (8 tests should pass)

Step 5: Inspect experiments in MLflow.

mlflow ui --backend-store-uri sqlite:///mlflow.db then open http://localhost:5000 to compare runs and see the registered model voltcast-forecaster.

Step 6: Start the prediction API.

uvicorn app:app --port 8000 then open http://localhost:8000/api/docs (Figure M.1). Use GET /api/sample-request to obtain a ready-made input, then paste it into POST /api/predict and click Execute.

Step 7: Start the dashboard.

streamlit run dashboard/app.py then open http://localhost:8501 (Figures M.2–M.6). In the Forecast tab choose a date, start hour and horizon; the chart shows history, forecast, actual values and the 95% interval, with predicted energy, cost and peak hour. Other tabs show the data explorer, model performance, monitoring and pipeline diagram.

Step 8: Run everything with Docker (optional).

docker compose up --build starts the API (port 8000), dashboard (8501) and MLflow UI (5000).

Step 9: Retrain with new settings.

Edit configs/params.yaml (e.g. model hyper-parameters or split dates) and re-run python run_pipeline.py (or dvc repro). A new MLflow run and model version are created.

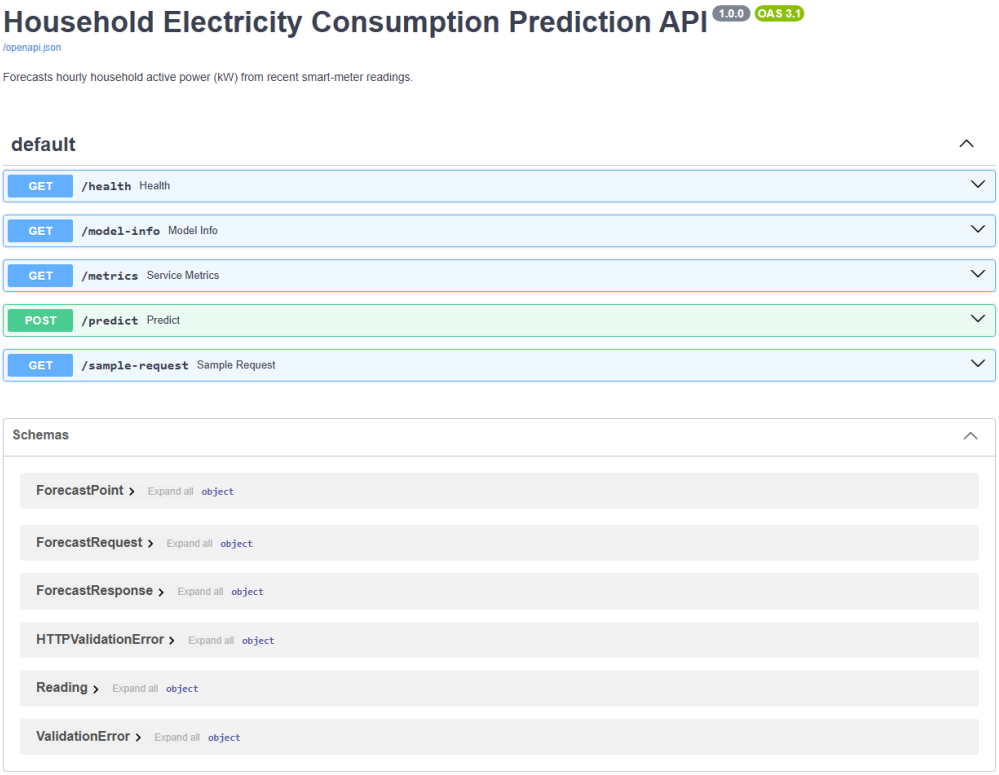


Figure M.1*: Interactive API documentation (Swagger UI) at /api/docs*

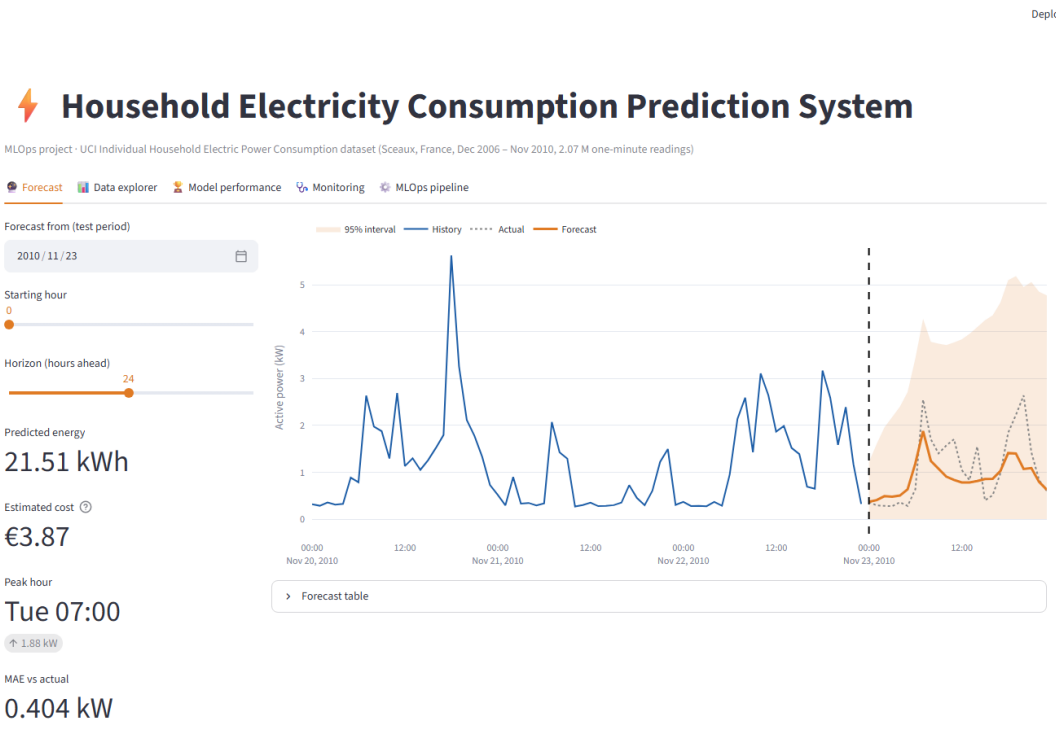


Figure M.2*: Streamlit dashboard – Forecast tab*

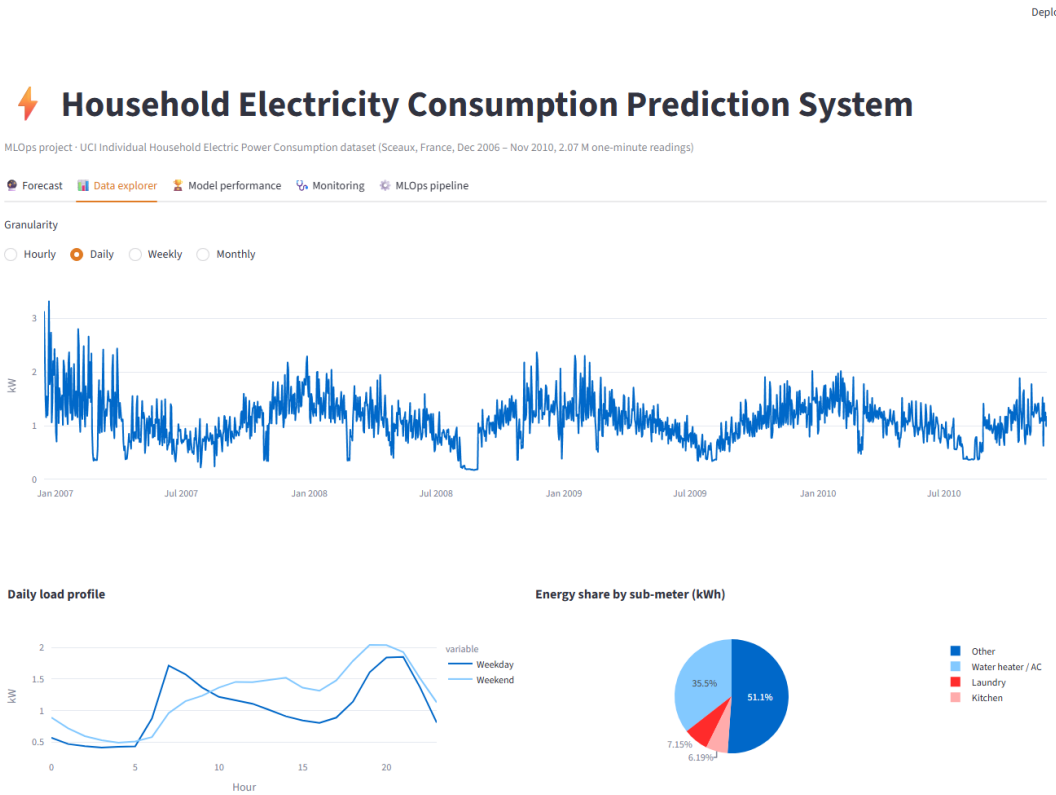


Figure M.3*: Streamlit dashboard – Data explorer tab*

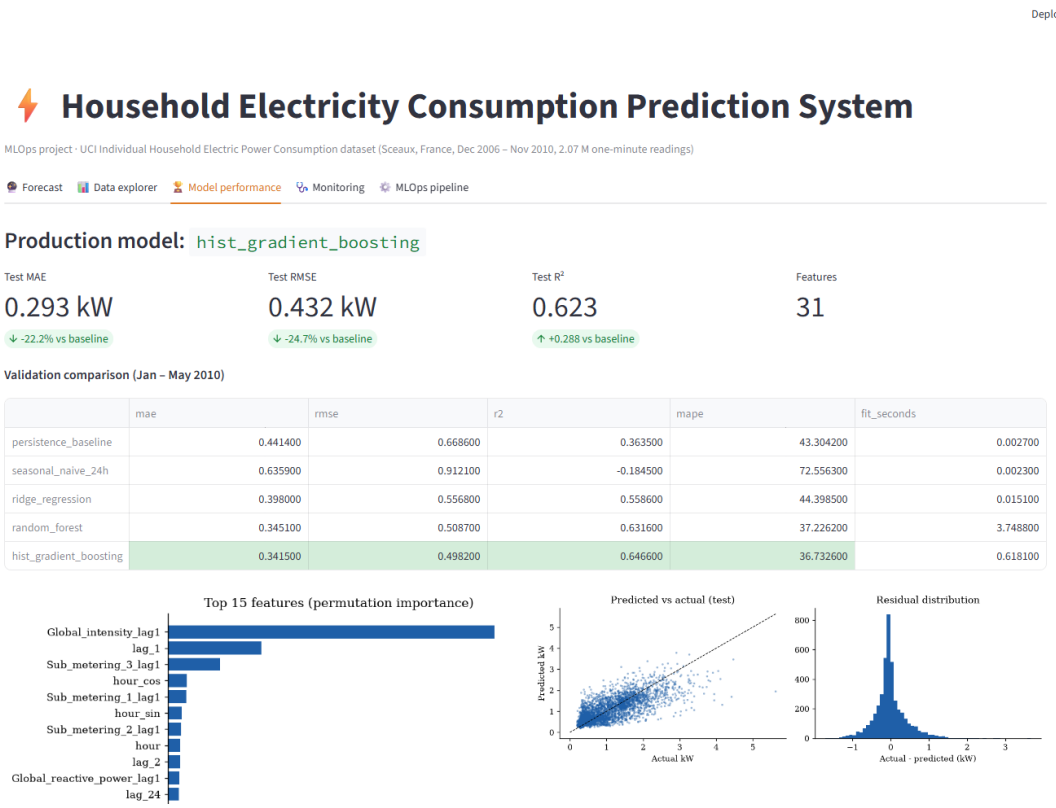


Figure M.4*: Streamlit dashboard – Model performance tab*

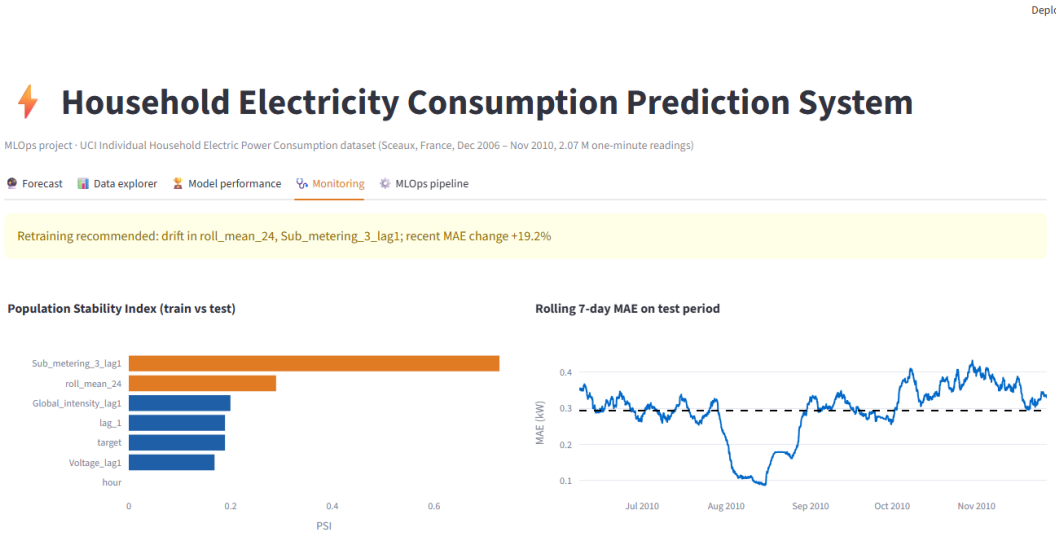
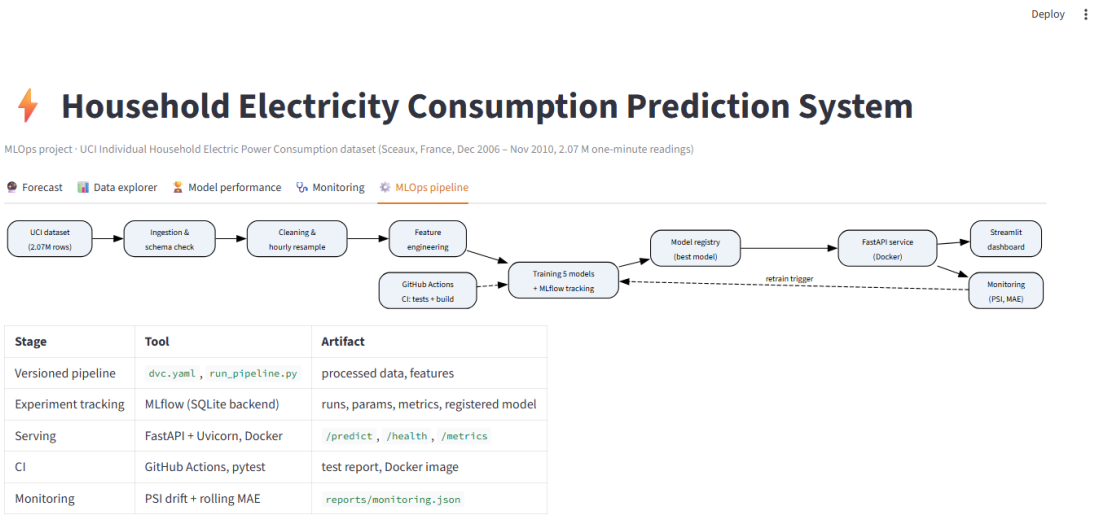


Figure M.5*: Streamlit dashboard – Monitoring tab*



Stage	Tool	Artifact
Versioned pipeline	`dvc.yaml` , `run_pipeline.py`	processed data, features
Experiment tracking	MLflow (SQLite backend)	runs, params, metrics, registered model
Serving	FastAPI + Uvicorn, Docker	`/predict` , `/health` , `/metrics`
CI	GitHub Actions, pytest	test report, Docker image
Monitoring	PSI drift + rolling MAE	`reports/monitoring.json`

Figure M.6*: Streamlit dashboard – MLOps pipeline tab*